

RAPPORT DE PROJET DE FIN D'ANNÉE

Soumis en vue de l'obtention du
DIPLÔME DE LICENCE EN SYSTÈMES EMBARQUÉS ET IOT
Parcours : Systèmes Embarqués et IoT

Système de Pointeuse Biométrique Connectée avec
Plateforme Web de Gestion Temps Réel (BioPulse)

Rédigé par :
MOHAMED DHIA CHAOUCH

Réalisé au sein de KSI



Superviseur professionnel : M. Ala JAOUACHI

Signature :

Année Universitaire : 2025-2026

Dédicaces

À mes chers parents, dont l'amour inconditionnel, les sacrifices et le soutien constant ont été ma source de force et d'inspiration tout au long de mon parcours académique et personnel.

À mes chers amis et collègues, pour leur présence rassurante, leurs encouragements inébranlables et les moments partagés qui ont enrichi cette expérience.

À toutes les personnes qui m'ont soutenu de près ou de loin, je vous exprime ma plus profonde gratitude et mon sincère respect.

Remerciements

Je tiens à exprimer ma profonde et sincère gratitude à toute l'équipe de l'entreprise **Kernel Solutions and Innovation (KSI)** pour leur accueil chaleureux, leur bienveillance et la mise à disposition d'un environnement de travail stimulant tout au long de cette période de stage.

Je souhaite adresser mes remerciements les plus respectueux à mon encadrant professionnel, **M. Ala JAOUACHI**, pour sa disponibilité, ses conseils avisés, son expertise technique et son accompagnement précieux lors du développement de ce projet.

Je remercie également l'ensemble des enseignants et membres du corps administratif de l'**Institut Supérieur des Technologies de l'Information et de la Communication (ISTIC)** pour la qualité de l'enseignement dispensé et leur soutien durant mon parcours universitaire.

Table des matières

Dédicaces	1
Remerciements	2
Introduction générale	7
1 Contexte du projet	8
1.1 Introduction	8
1.2 Organisme d'accueil	8
1.3 Présentation du projet	9
1.4 Étude de l'existant	9
1.4.1 Définition des systèmes de pointeuse biométrique	9
1.4.2 Historique et évolution des systèmes de pointage	9
1.4.3 Description de l'état actuel et limites du marché	10
1.4.4 Problématique	10
1.4.5 Solution proposée	11
1.5 Méthodologie de travail et planification	11
1.5.1 Choix de la méthodologie Agile + Kanban	11
1.5.2 Planification des tâches	11
1.6 Conclusion	12
2 Étude du projet	13
2.1 Introduction	13
2.2 Analyse des besoins	13
2.2.1 Besoins fonctionnels	13
2.2.2 Besoins non-fonctionnels	14
2.3 Modélisation générale et conception du système	14
2.3.1 Diagramme d'architecture globale	14
2.3.2 Diagramme de cas d'utilisation général (UML)	15
2.4 Conception logicielle du système	16
2.4.1 Diagramme d'activité du firmware ESP32	16
2.4.2 Diagramme d'activité du serveur Backend Node.js / Express	17
2.4.3 Diagramme d'activité de l'application Web React	18
2.5 Conclusion	19

3	Réalisation du système	20
3.1	Introduction	20
3.2	Environnement de travail	20
3.2.1	Environnement matériel	20
3.2.2	Environnement logiciel	21
3.3	Schéma de câblage électronique et affectation des broches	22
3.4	Architecture logicielle et langages de développement	23
3.5	Réalisation des modules et développement des interfaces	23
3.5.1	Module Embarqué (Firmware ESP32 & Interface LCD)	23
3.5.2	Module Web (Plateforme Backend & Interface React)	24
3.6	Conclusion	26
4	Extension du projet et évaluation	27
4.1	Introduction	27
4.2	Module d'extension et de résilience du système	27
4.2.1	Découverte automatique UDP du serveur central	27
4.2.2	Bascule réseau intelligente (Failover Ethernet / Wi-Fi)	28
4.2.3	Synchronisation temporelle NTP	28
4.3	Évaluation et validation des performances	28
4.3.1	Tests de reconnaissance biométrique	28
4.3.2	Tests de réactivité et latence réseau	28
4.4	Conclusion du chapitre 4	29
	Conclusion générale et perspectives	30

Table des figures

1.1	Logo de KSI.	8
1.2	Évolution historique des systèmes de pointage de présence.	10
1.3	Méthodologie Agile Kanban.	11
1.4	Planification des travaux sur le tableau Kanban.	12
2.1	Diagramme d'architecture globale du système BioPulse.	15
2.2	Diagramme de cas d'utilisation général (UML).	16
2.3	Diagramme d'activité du firmware ESP32.	17
2.4	Diagramme d'activité du serveur Backend Node.js / Express.	18
2.5	Diagramme d'activité de l'application Web React.	19
3.1	Schéma de câblage électrique et synoptique d'assemblage du terminal BioPulse. . .	23
3.2	Rendu de l'interface LCD du terminal matériel BioPulse lors d'un pointage valide. .	24
3.3	Interface de connexion sécurisée au portail d'administration BioPulse.	24
3.4	Tableau de bord principal de suivi des présences en temps réel.	25
3.5	Interface de gestion des employés et suivi de l'enrôlement des empreintes.	25

Liste des tableaux

- 3.1 Synthèse des composants matériels du terminal BioPulse. 21
- 3.2 Synthèse des outils logiciels et frameworks utilisés. 21
- 3.3 Affectation détaillée des broches GPIO du terminal BioPulse. 22

Introduction générale

Dans un monde professionnel moderne en pleine transformation numérique, la gestion automatisée du temps de présence est devenue un enjeu majeur pour les entreprises et institutions. Les méthodes traditionnelles basées sur le pointage manuel sur papier ou des fiches de présence présentent des vulnérabilités, notamment les risques de fraude, les erreurs de saisie et la difficulté de centraliser les données en temps réel. Pour répondre à ces défis, l'intégration des technologies de l'Internet des Objets (IoT) et de la biométrie offre une alternative robuste, précise et automatisée.

Ce projet s'inscrit dans ce contexte et vise à concevoir et réaliser une pointeuse biométrique intelligente (basée sur un microcontrôleur ESP32 et un capteur d'empreintes digitales) connectée à une plateforme web de gestion en temps réel (développée avec Node.js, Express, MongoDB et React). Le système permet d'authentifier les employés par leurs empreintes digitales, de communiquer de manière résiliente via Wi-Fi ou Ethernet (protocole UDP et HTTP), et d'offrir un tableau de bord analytique complet dédié exclusivement au suivi rigoureux des présences, des retards et des absences.

Ce rapport est structuré en quatre chapitres principaux :

- Le **Premier Chapitre** présente le cadre général du projet, l'organisme d'accueil KSI, la problématique, la solution proposée ainsi que la méthodologie de travail adoptée.
- Le **Deuxième Chapitre** est consacré à l'étude du projet, détaillant l'analyse des besoins fonctionnels et non-fonctionnels, la modélisation UML et la conception logicielle/matérielle.
- Le **Troisième Chapitre** décrit la réalisation du système, incluant l'environnement de développement, le câblage électronique des composants matériels, la programmation du firmware ESP32 et le développement de la plateforme web.
- Le **Quatrième Chapitre** traite d'un module d'extension et de résilience du système (découverte automatique UDP du serveur, gestion des erreurs de connexion, bascule réseau Ethernet/Wi-Fi et synchronisation), suivi de la présentation des résultats obtenus.

Chapitre 1

Contexte du projet

1.1 Introduction

Dans ce premier chapitre, nous présentons le cadre général dans lequel s'inscrit notre projet de fin d'année. Nous commençons par une présentation détaillée de l'entreprise d'accueil, **Kernel Solutions and Innovation (KSI)**. Ensuite, nous exposons la situation problématique ayant motivé ce travail, analysons les solutions existantes à travers une étude de l'état de l'art, et décrivons la solution proposée. Enfin, nous présentons la méthodologie de travail adoptée (Agile / Kanban) ainsi que la planification temporelle des tâches réalisées.

1.2 Organisme d'accueil

Kernel Solutions and Innovation (KSI) est une société de services et d'ingénierie informatique spécialisée dans le développement de solutions technologiques innovantes. L'entreprise accompagne aussi bien les entreprises commerciales que les institutions en leur fournissant des services d'ingénierie logicielle sur mesure, d'intégration de systèmes IoT et de conseil technologique.



FIGURE 1.1 – Logo de KSI.

Grâce à une équipe d'ingénieurs qualifiés, KSI se distingue par sa capacité à concevoir des solutions performantes, évolutives et adaptées aux besoins spécifiques de ses clients. Les principaux

domaines d'activité de l'entreprise comprennent :

1. **Développement de logiciels sur mesure** : Conception d'applications web, mobiles et embarquées intuitives et performantes.
2. **Services de maintenance et de support** : Suivi post-déploiement, mises à jour technologiques et assistance technique continue.
3. **Conseil technologique et IoT** : Accompagnement stratégique, intégration de systèmes matériels/logiciels et optimisation des infrastructures IT.

1.3 Présentation du projet

L'objectif de ce projet est de concevoir et de développer un système complet et intelligent de gestion du temps de présence, baptisé **BioPulse**. Ce système combine un terminal matériel autonome (pointeuse biométrique basée sur le microcontrôleur ESP32, un capteur d'empreintes digitales Adafruit, un écran graphique LCD ILI9341, un pavé numérique et un double contrôleur réseau Ethernet/Wi-Fi) et une plateforme web centralisée (développée avec Node.js, Express, MongoDB et React).

Le terminal enregistre et authentifie localement les empreintes digitales des employés en temps réel, transmet les pointages vers le serveur via un protocole réseau sécurisé et résilient, et permet une gestion transparente des présences, des retards et des absences à travers une interface web moderne et réactive.

1.4 Étude de l'existant

L'étude de l'état de l'art constitue une étape préalable indispensable afin d'analyser les technologies existantes dans le domaine de l'identification et du suivi de présence, d'en identifier les limites et de justifier nos choix d'ingénierie.

1.4.1 Définition des systèmes de pointeuse biométrique

Une pointeuse biométrique est un dispositif électronique d'authentification et de suivi du temps de présence qui exploite les empreintes digitales pour enregistrer automatiquement l'heure d'arrivée et de départ des collaborateurs. Contrairement aux méthodes conventionnelles reposant sur des cartes ou fiches papier, l'empreinte digitale offre une preuve d'identité irréfutable et infalsifiable. Les caractéristiques de l'empreinte sont numérisées sous forme de gabarit (template) et stockées de manière sécurisée pour être comparées en quelques millisecondes lors de chaque pointage.

1.4.2 Historique et évolution des systèmes de pointage

Le pointage du temps de travail a connu plusieurs générations technologiques :

- **Les horloges mécaniques (fin XIXe - XXe siècle)** : Pointage manuel sur cartes carton perforées.
- **Les systèmes à cartes magnétiques et RFID (années 1980 - 1990)** : Automatisation du stockage informatique, mais vulnérabilité au prêt de badge entre collègues ("buddy punching").
- **Les pointeuses biométriques et IoT (depuis les années 2000)** : Démocratisation des capteurs d'empreintes optiques et capacitifs couplés à des microcontrôleurs réseau, apportant une fiabilité totale et une centralisation des données en temps réel.

TECHNOLOGICAL EVOLUTION OF TIME ATTENDANCE SYSTEMS

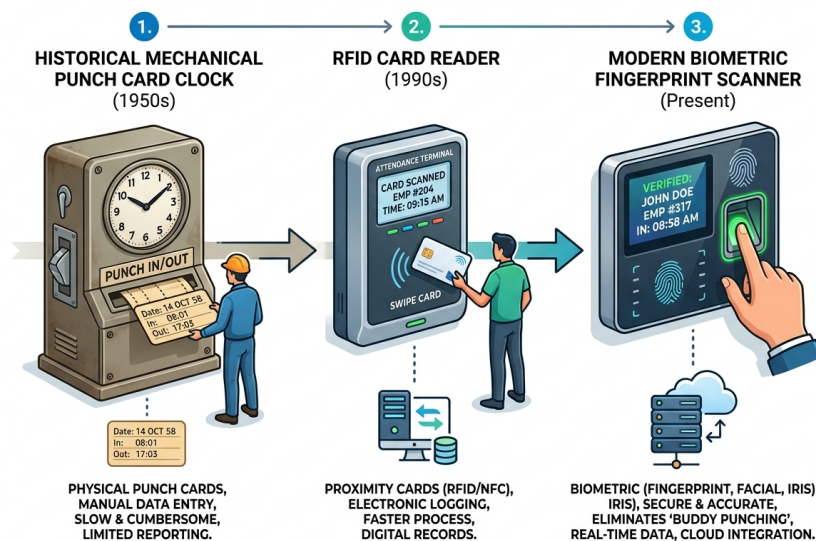


FIGURE 1.2 – Évolution historique des systèmes de pointage de présence.

1.4.3 Description de l'état actuel et limites du marché

De nombreuses pointeuses commerciales existent sur le marché. Néanmoins, l'analyse des offres actuelles met en évidence des contraintes majeures :

- **Coûts élevés et systèmes fermés** : Les pointeuses propriétaires imposent souvent des licences d'utilisation onéreuses et des logiciels rigides impossibles à personnaliser.
- **Dépendance au réseau central** : Beaucoup de pointeuses ordinaires cessent de fonctionner en cas de rupture de communication Wi-Fi ou d'indisponibilité du serveur local.

1.4.4 Problématique

Au vu de cet état des lieux, la problématique de ce projet s'articule autour de trois verrous techniques majeurs :

1. Comment supprimer la fraude et les erreurs manuelles lors de la saisie des présences ?
2. Comment développer une solution matérielle et logicielle économique, sur mesure et parfaitement adaptée aux besoins de l'entreprise ?

3. Comment assurer la continuité de la communication réseau entre la pointeuse et la plateforme web ?

1.4.5 Solution proposée

Pour répondre à ces enjeux, nous proposons la réalisation de la solution **BioPulse**, qui s'appuie sur deux piliers complémentaires :

- **Un terminal biométrique intelligent** : Basé sur le microcontrôleur ESP32, un capteur d'empreintes Adafruit, un écran LCD ILI9341, un pavé numérique et un double contrôleur réseau (Ethernet W5500 / Wi-Fi). Il intègre la découverte automatique UDP du serveur pour une communication réseau réactive et fluide.
- **Une plateforme Web de gestion temps réel** : Développée avec Node.js, Express, MongoDB et React, offrant un tableau de bord ergonomique pour le suivi des présences, la détection automatique des retards/absences et la gestion des employés.

1.5 Méthodologie de travail et planification

1.5.1 Choix de la méthodologie Agile + Kanban

Afin de mener à bien ce projet dans le respect des délais tout en conservant une grande flexibilité d'adaptation face aux exigences techniques matérielles et logicielles, nous avons adopté la méthodologie **Agile** combinée avec l'outil visuel **Kanban**.

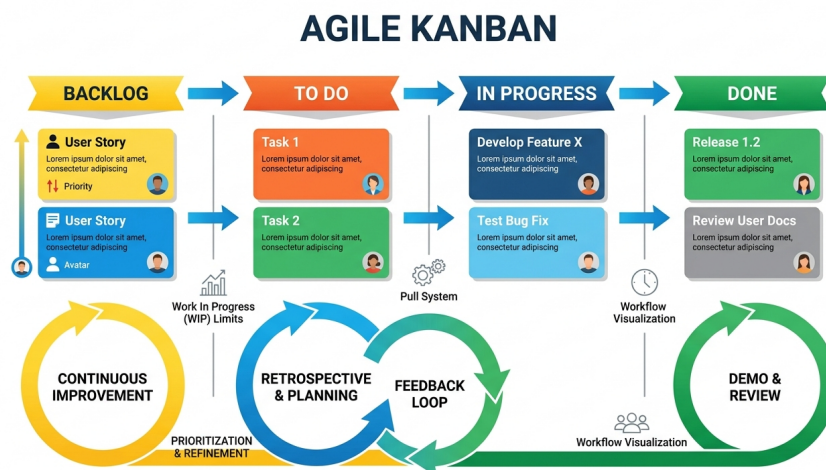


FIGURE 1.3 – Méthodologie Agile Kanban.

Cette approche hybride privilégie le développement itératif, la livraison continue des fonctionnalités et une visibilité transparente sur l'avancement des tâches grâce à trois colonnes principales : **À faire (To Do)**, **En cours (In Progress)** et **Terminé (Done)**.

1.5.2 Planification des tâches

Le projet a été découpé en quatre grandes phases itératives (Sprints) :

- **Sprint 1 - Étude et analyse des besoins** : Étude théorique, choix des composants matériels et modélisation des besoins (diagrammes UML).
- **Sprint 2 - Développement du terminal embarqué** : Assemblage matériel, câblage, programmation du firmware ESP32 (capteur d'empreintes, écran LCD, pavé numérique).
- **Sprint 3 - Développement de la plateforme web** : Création de la base de données MongoDB, développement de l'API REST backend (Express) et de l'interface React.
- **Sprint 4 - Communication réseau, tests et intégration** : Implémentation de la découverte UDP, configuration du double contrôleur Ethernet/Wi-Fi, tests d'intégration et validation globale.

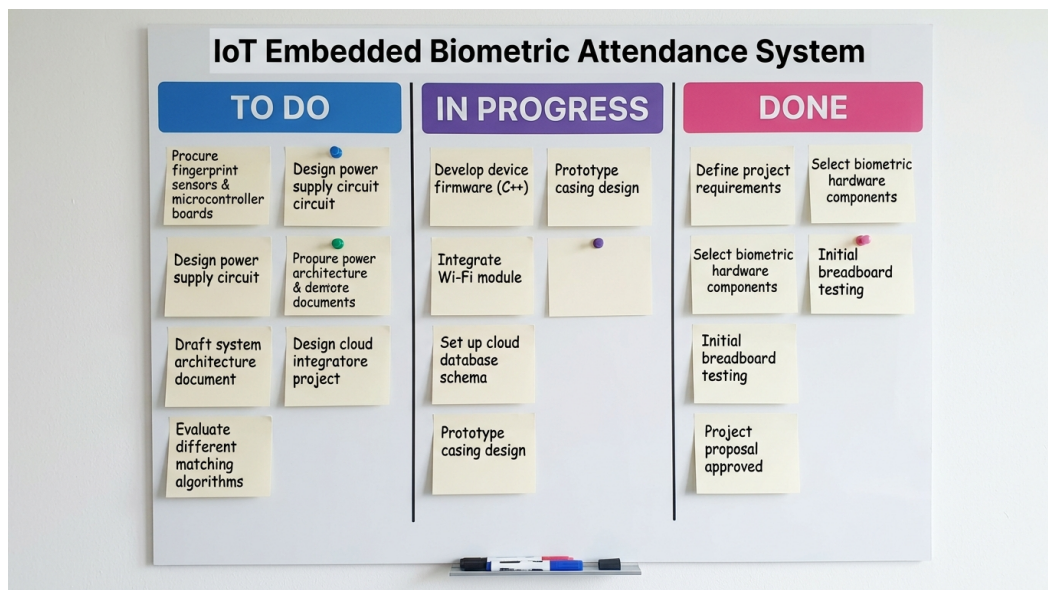


FIGURE 1.4 – Planification des travaux sur le tableau Kanban.

1.6 Conclusion

Ce premier chapitre a posé les fondations de notre travail en présentant le cadre de notre projet chez KSI, les objectifs du système BioPulse, la problématique du pointage et la solution adoptée. Nous avons également détaillé la démarche méthodologique Agile-Kanban qui a guidé nos travaux.

Dans le chapitre suivant, nous aborderons l'étude approfondie du projet à travers l'analyse des besoins fonctionnels et non-fonctionnels ainsi que la modélisation UML.

Chapitre 2

Étude du projet

2.1 Introduction

Ce deuxième chapitre est consacré à l'étude détaillée et à la conception de notre système **BioPulse**. Nous commençons par une analyse rigoureuse des besoins fonctionnels et non-fonctionnels du système. Ensuite, nous présentons la modélisation générale à travers le diagramme d'architecture globale et le diagramme de cas d'utilisation (UML). Enfin, nous détaillons la conception logicielle du système en décrivant les diagrammes d'activité du firmware ESP32, du serveur backend Node.js/Express et de l'interface web React.

2.2 Analyse des besoins

Afin de répondre aux objectifs de notre projet et de définir avec précision le périmètre du système, nous identifions les besoins fonctionnels et non-fonctionnels.

2.2.1 Besoins fonctionnels

Le système de pointeuse biométrique **BioPulse** doit répondre aux exigences fonctionnelles suivantes :

- **Authentification et Pointage Biométrique :**
 - Enregistrer l'empreinte digitale des nouveaux employés et lui attribuer un identifiant unique (ID).
 - Reconnaître et authentifier les empreintes digitales en temps réel lors du pointage d'entrée ou de sortie.
 - Afficher le résultat de l'authentification (succès avec le nom de l'employé ou échec) sur l'écran LCD ILI9341 et émettre un signal sonore/visuel.
- **Gestion des Employés et des Départements :**
 - Permettre aux administrateurs de créer, modifier, consulter et supprimer les fiches des employés via l'interface web.
 - Associer chaque employé à un département spécifique et lui attribuer un rôle.
- **Suivi et Gestion des Présences :**

- Enregistrer automatiquement la date, l'heure exacte et le statut (Présent, En Retard, Départ Anticipé, Absent) lors de chaque pointage.
- Calculer automatiquement les retards et les heures de présence journalières.
- **Communication Réseau et Synchronisation :**
 - Transmettre instantanément les données de pointage depuis l'ESP32 vers la plateforme web via le réseau local.
 - Permettre la découverte automatique de l'adresse IP du serveur backend grâce au protocole de diffusion UDP.
 - Gérer la bascule automatique du réseau entre l'interface Ethernet W5500 et le Wi-Fi pour maintenir la communication.

2.2.2 Besoins non-fonctionnels

Afin de garantir une haute qualité de service, le système doit satisfaire aux contraintes non-fonctionnelles suivantes :

- **Temps de réponse et Réactivité (Latence) :** L'authentification de l'empreinte digitale et l'affichage du résultat sur l'écran LCD doivent s'effectuer en moins d'une seconde (< 1 s). La transmission du pointage vers la plateforme web doit être quasi-instantanée (< 200 ms).
- **Sécurité et Confidentialité :** L'accès au panneau d'administration web est protégé par un système d'authentification robuste (tokens JWT et mots de passe hachés avec Bcrypt). Les empreintes ne sont pas stockées sous forme d'images brutes mais sous forme de gabarits numériques (templates) non réversibles.
- **Résilience et Disponibilité :** Le terminal matériel assure la continuité de service grâce à la détection automatique de la connexion et à la bascule automatique Ethernet/Wi-Fi.
- **Ergonomie et Usabilité :** L'interface web doit être intuitive, réactive (design responsive avec Tailwind/CSS) et offrir une visualisation claire des indicateurs sur le tableau de bord.
- **Maintenabilité et Évolutivité :** Le code source du firmware ESP32 et de la plateforme Node.js/React doit être modulaire et bien documenté pour faciliter l'ajout de nouveaux modules matériels ou logiciels à l'avenir.

2.3 Modélisation générale et conception du système

Cette section présente la modélisation globale du système à travers les représentations d'architecture et de cas d'utilisation UML, explicitant le fonctionnement interactif entre la pointeuse matérielle, le réseau et la plateforme web.

2.3.1 Diagramme d'architecture globale

Le diagramme d'architecture globale (Figure 2.1) illustre l'organisation en couches du système BioPulse :

- **La couche matérielle (Terminal ESP32)** : Regroupe le microcontrôleur ESP32, le capteur biométrique d'empreintes digitales Adafruit, l'écran graphique LCD ILI9341, le pavé numérique et les contrôleurs réseau (Ethernet W5500 et Wi-Fi).
- **La couche réseau et communication** : Assure la transmission des paquets de pointage et la découverte automatique par diffusion UDP du serveur backend sur le réseau local.
- **La couche serveur Backend (Node.js / Express / MongoDB)** : Gère les API REST, traite la logique métier des présences, authentifie les administrateurs et enregistre les données dans MongoDB.
- **La couche utilisateur Frontend (React)** : Offre une interface web réactive pour la visualisation en temps réel du tableau de bord, la gestion des employés et la configuration du système.

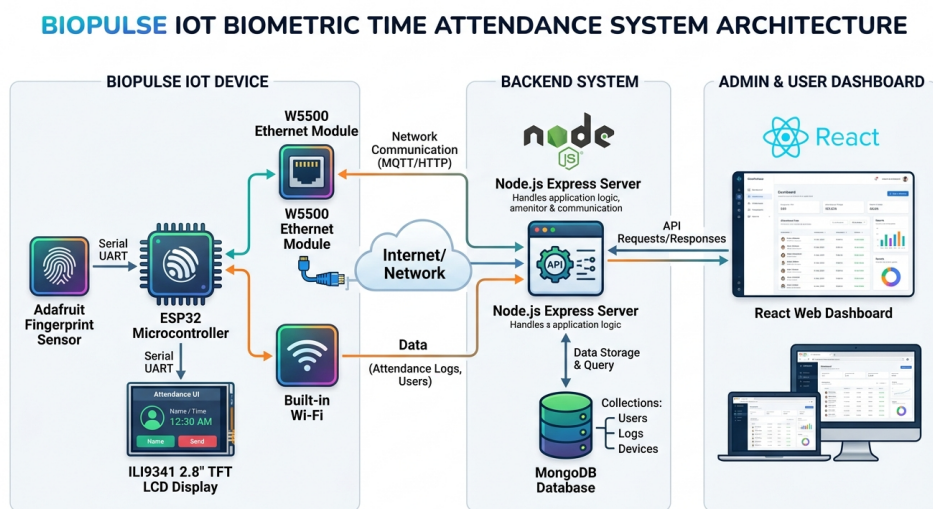


FIGURE 2.1 – Diagramme d'architecture globale du système BioPulse.

2.3.2 Diagramme de cas d'utilisation général (UML)

Le diagramme de cas d'utilisation (Figure 2.2) formalise les fonctionnalités offertes par le système et les rôles des différents acteurs qui interagissent avec lui.

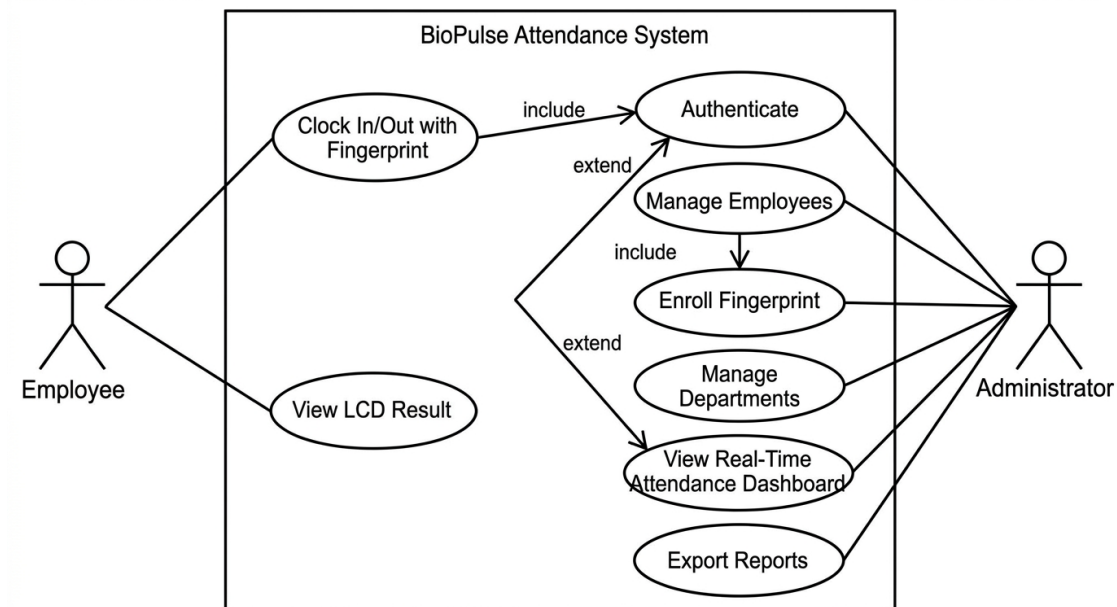


FIGURE 2.2 – Diagramme de cas d'utilisation général (UML).

Comme l'illustre la Figure 2.2, nous distinguons deux acteurs principaux :

- **L'Employé** : Interagit directement avec le terminal matériel. Ses cas d'utilisation incluent :
 - *Pointer la présence* : Placer son empreinte sur le capteur biométrique pour valider son arrivée ou son départ.
 - *Consulter la confirmation* : Observer la notification visuelle (nom de l'employé et heure) sur l'écran LCD.
- **L'Administrateur** : Interagit avec la plateforme Web. Ses cas d'utilisation incluent :
 - *S'authentifier* : Accéder à la plateforme via login/mot de passe sécurisé.
 - *Gérer les employés* : Créer, consulter, modifier ou supprimer un compte employé et l'associer à un département.
 - *Enrôler les empreintes* : Associer un nouvel identifiant biométrique à la fiche d'un employé.
 - *Consulter les présences* : Examiner le tableau de bord temps réel, filtrer les retards, les absences et exporter les historiques.
 - *Gérer le système et les départements* : Organiser la structure de l'entreprise et surveiller l'état de connexion du terminal.

2.4 Conception logicielle du système

Cette section détaille les diagrammes d'activité UML illustrant les processus dynamiques régissant le firmware ESP32, le serveur backend et l'application Web frontend.

2.4.1 Diagramme d'activité du firmware ESP32

Le déroulement du firmware ESP32 (Figure 2.3) s'effectue en deux phases principales. Durant la phase d'initialisation (Setup), l'ESP32 démarre la liaison série, configure le bus I2C pour le

capteur d'empreintes digitales Adafruit, initialise l'écran LCD ILI9341, le pavé numérique et tente la connexion réseau via Ethernet W5500 ou Wi-Fi. Il lance ensuite le service de découverte UDP pour identifier l'adresse IP du serveur central.

Après le setup, l'ESP32 s'exécute dans une boucle infinie (Loop). À chaque cycle, il scrute la présence d'un doigt sur le capteur biométrique : lors de la détection d'une empreinte, il la numérise et la compare aux gabarits enregistrés. Si l'empreinte est valide, l'ESP32 formate les données de pointage (ID employé et horodatage), les émet via une requête HTTP/UDP vers le serveur, affiche un message de bienvenue personnalisé sur l'écran LCD et émet un signal sonore de confirmation.

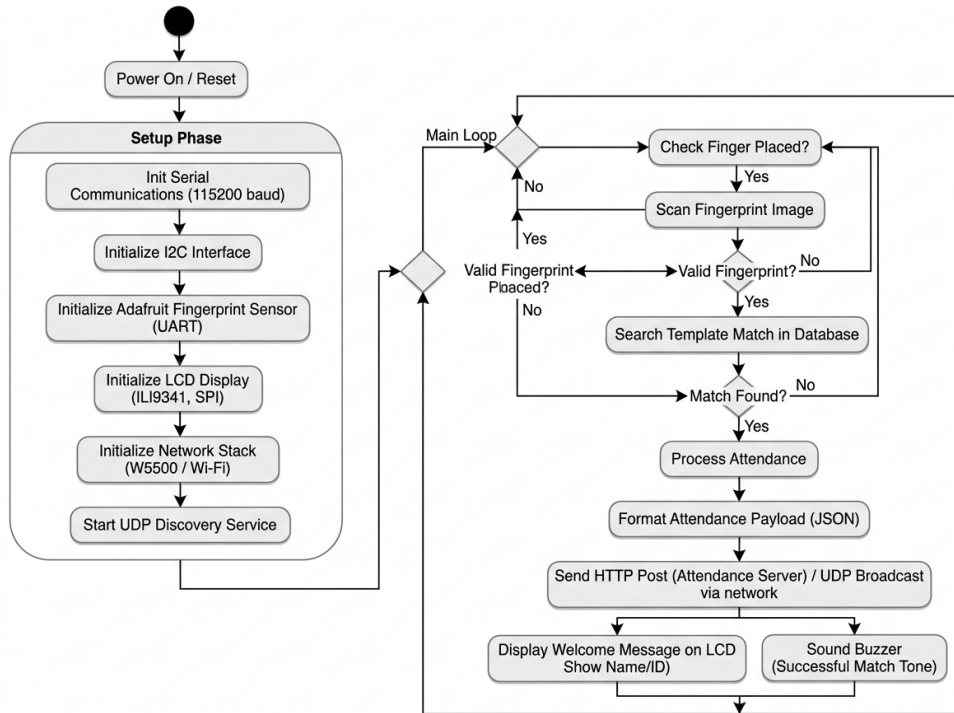


FIGURE 2.3 – Diagramme d'activité du firmware ESP32.

2.4.2 Diagramme d'activité du serveur Backend Node.js / Express

Le serveur backend (Figure 2.4) agit comme le cœur décisionnel du système. Lorsqu'il reçoit une requête de pointage en provenance de la pointeuse ESP32, il effectue la vérification de l'identifiant employé dans la base de données MongoDB. Il calcule automatiquement le statut de présence en fonction des horaires de travail configurés (Présent, En Retard, Départ Anticipé), enregistre la transaction horodatée dans la collection des présences, et renvoie une réponse JSON de confirmation au terminal matériel. Simultanément, le serveur émet une notification temps réel vers le tableau de bord web React pour actualiser instantanément les statistiques de présence.

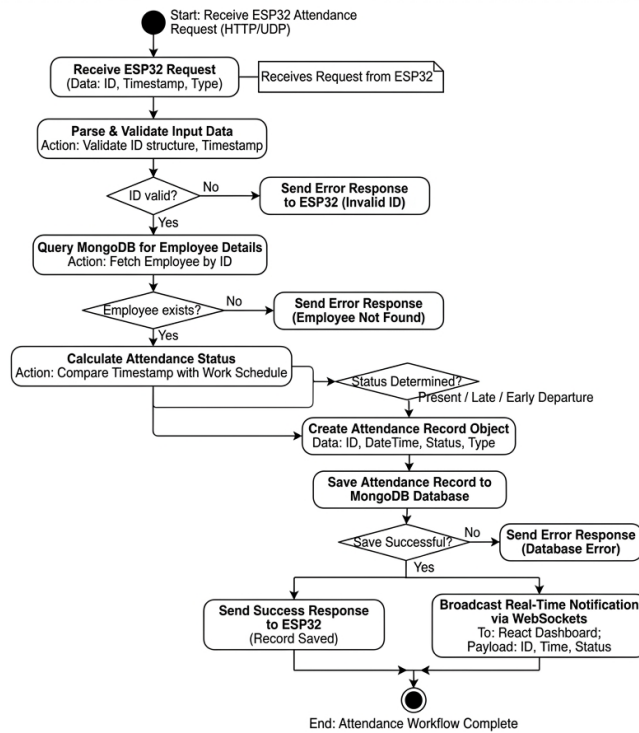


FIGURE 2.4 – Diagramme d'activité du serveur Backend Node.js / Express.

2.4.3 Diagramme d'activité de l'application Web React

L'application Web frontend (Figure 2.5) permet aux administrateurs de superviser l'ensemble du système. Après une authentification sécurisée par jeton JWT, l'administrateur accède au tableau de bord principal. L'interface se synchronise dynamiquement avec le backend pour afficher en temps réel les pointeuses connectées, la liste des employés présents/absents/en retard, et permet d'exécuter les opérations de gestion (création de fiches employés, attribution de départements et exportation des rapports de présence).

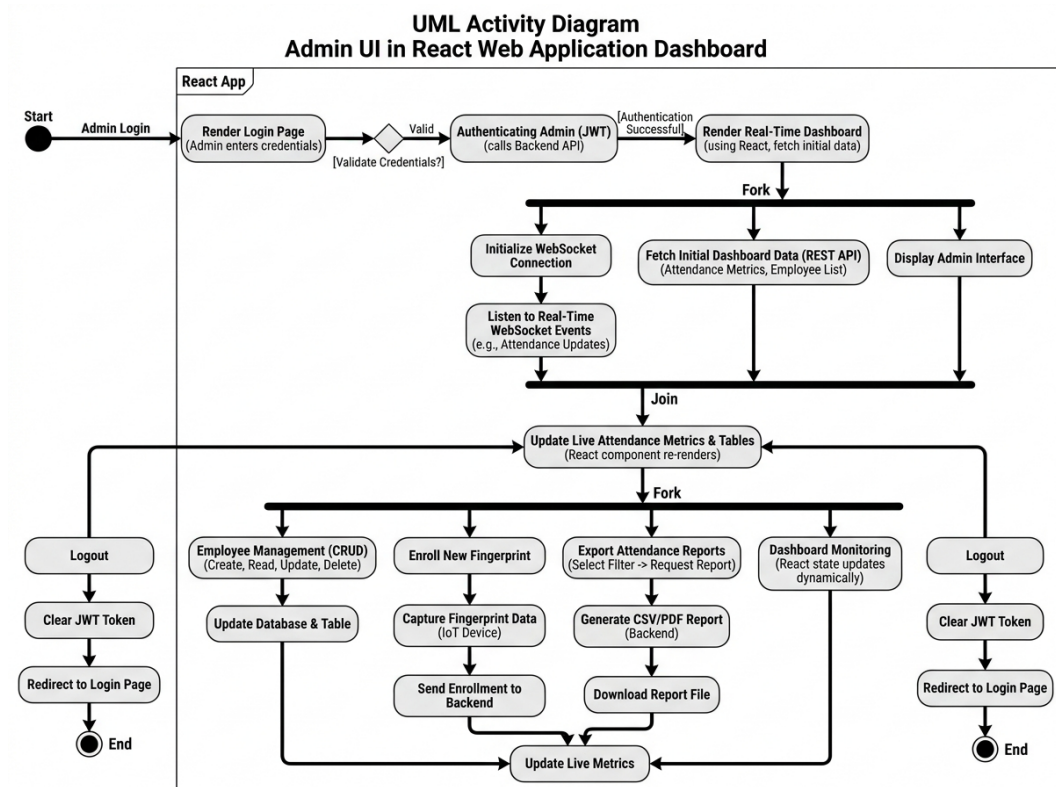


FIGURE 2.5 – Diagramme d'activité de l'application Web React.

2.5 Conclusion

Ce deuxième chapitre a présenté l'étude détaillée du système **BioPulse**. Nous avons défini l'ensemble des exigences fonctionnelles et non-fonctionnelles, puis modélisé l'architecture globale, les cas d'utilisation UML ainsi que les diagrammes d'activité régissant le fonctionnement de l'ESP32, du serveur backend et de l'interface React.

Ces modèles théoriques et fonctionnels nous servent de guide pour la phase de réalisation technique qui fera l'objet du chapitre suivant.

Chapitre 3

Réalisation du système

3.1 Introduction

Ce troisième chapitre décrit la mise en œuvre pratique et technique de notre système **Bio-Pulse**. Nous commençons par détailler l’environnement de travail en présentant la liste exhaustive des composants matériels et logiciels utilisés. Ensuite, nous exposons le schéma de câblage électronique ainsi que l’affectation précise des broches GPIO sous forme d’un tableau récapitulatif. Puis, nous présentons l’architecture logicielle et les langages de programmation adoptés. Enfin, nous détaillons la réalisation concrète des modules embarqué (firmware ESP32, écran LCD) et web (backend Node.js, interface React).

3.2 Environnement de travail

Cette section présente le cadre technique de développement, regroupant l’environnement matériel et logiciel constituant notre système.

3.2.1 Environnement matériel

Le terminal matériel autonome repose sur l’assemblage de composants électroniques spécifiquement sélectionnés pour leur fiabilité, leur faible coût et leurs performances :

TABLE 3.1 – Synthèse des composants matériels du terminal BioPulse.

Composant	Spécifications	Rôle dans le projet
ESP32-WROOM-32	Microcontrôleur 32 bits, 240 MHz, SRAM 520 KB	Unité centrale de traitement, exécution du firmware C++ et gestion du réseau.
Capteur Adafruit Fingerprint	Module biométrique optique (UART)	Capture, numérisation et authentification locale des empreintes digitales.
Écran LCD ILI9341 (TFT 2.8")	Écran couleur 240 × 320 px (SPI)	Affichage de l'horloge, des messages de bienvenue et des retours visuels.
Module Ethernet W5500	Contrôleur SPI TCP/IP matériel	Assure la connectivité filaire résiliente du terminal sur le réseau local.
Pavé numérique Keypad 4x4	Clavier matriciel à 16 touches	Saisie d'identifiants manuels et navigation dans les menus d'administration.
LEDs de statut (Verte/Rouge)	Diodes électroluminescentes GPIO	Signalisation lumineuse instantanée du succès ou de l'échec de pointage.
Buzzer Piézoélectrique	Avertisseur sonore 5V DC	Émission de signaux audio (bip de validation, bips d'erreur).
Régulateur LM2596	Convertisseur Step-Down DC-DC 5V	Conversion de l'alimentation 12V externe vers un 5V/3.3V régulé et sécurisé.

3.2.2 Environnement logiciel

La réalisation logicielle du projet s'est appuyée sur une suite d'outils de développement modernes :

TABLE 3.2 – Synthèse des outils logiciels et frameworks utilisés.

Outil / Framework	Catégorie	Usage et Fonction
Arduino IDE	Environnement de développement C++	Programmation, compilation et téléversement du firmware sur l'ESP32.
Visual Studio Code	Éditeur de code source	Développement global des applications web (Node.js, Express, React).
Node.js & Express	Runtime & Middleware Server	Traitement des API REST, gestion de la logique métier et diffusion WebSockets.
MongoDB & Compass	Base de données NoSQL	Stockage et modélisation des fiches employés, départements et historiques.
React & Vite	Framework Web Frontend	Conception du tableau de bord d'administration réactif et dynamique.

3.3 Schéma de câblage électronique et affectation des broches

Le raccordement électrique des différents composants matériels autour du microcontrôleur ESP32-WROOM-32 a été conçu avec soin pour optimiser l'allocation des broches GPIO et garantir la stabilité des communications sur bus SPI et UART.

Le Tableau 3.3 présente la matrice complète d'affectation des broches GPIO utilisées dans le terminal matériel **BioPulse** :

TABLE 3.3 – Affectation détaillée des broches GPIO du terminal BioPulse.

Composant matériel	Broche / Signal	Broche ESP32	Description / Bus
Alimentation LM2596	OUT+ / OUT-	VIN / GND	Tension régulée 5V DC & Masse commune
Écran LCD ILI9341	CS / DC / RST	GPIO 5 / 2 / 4	Bus SPI (Chip Select, Data/Cmd, Reset)
Écran LCD ILI9341	MOSI / MISO / SCK	GPIO 23 / 19 / 18	Bus SPI Matériel (Data In/Out, Clock)
Module Ethernet W5500	ETH_CS / ETH_RST	GPIO 22 / 15	Bus SPI (CS Ethernet & Reset W5500)
Module Ethernet W5500	MOSI / MISO / SCK	GPIO 23 / 19 / 18	Bus SPI Matériel (Partagé avec l'écran LCD)
Capteur Adafruit	RX / TX	GPIO 16 / 17	Liaison Série UART2 (115200 baud)
Keypad 4x4 (Lignes)	R1 / R2 / R3 / R4	GPIO 32 / 33 / 25 / 26	Matrice de saisie - Lignes 1 à 4
Keypad 4x4 (Colonnes)	C1 / C2 / C3 / C4	GPIO 27 / 14 / 13 / 34	Matrice de saisie - Colonnes 1 à 4
LED Jaune	Anode (+)	GPIO 12	Témoin d'attente et état de connexion
LED Verte	Anode (+)	GPIO 3 (RX0)	Témoin de pointage réussi (Valide)
LED Rouge	Anode (+)	GPIO 1 (TX0)	Témoin de pointage refusé (Échec)
Buzzer Piézo	Signal (+)	GPIO 21	Signalisation sonore (Impulsions PWM)

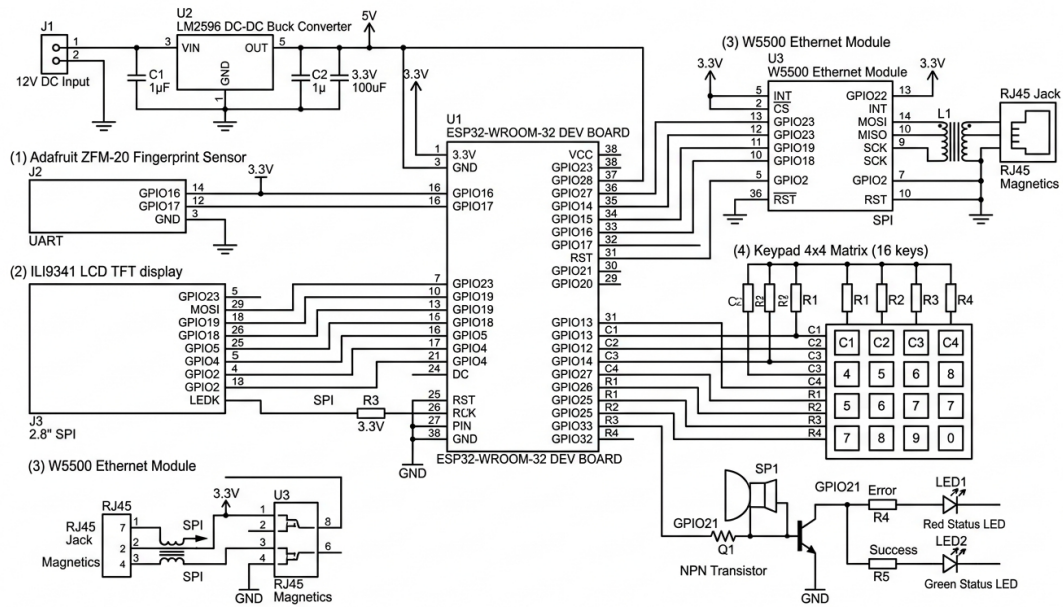


FIGURE 3.1 – Schéma de câblage électrique et synoptique d'assemblage du terminal BioPulse.

3.4 Architecture logicielle et langages de développement

Le développement de la solution **BioPulse** s'appuie sur une séparation claire des responsabilités entre la couche matérielle embarquée et la couche applicative web :

- **Langage C/C++ (Arduino / ESP32)** : Utilisé pour le codage du firmware embarqué, gérant les interruptions matérielles, l'exécution des bibliothèques biométriques Adafruit, le rendu graphique LCD ILI9341 et la communication sockets HTTP/UDP.
- **JavaScript (ES6+ / Node.js)** : Langage principal côté serveur pour concevoir l'API REST Express, valider la logique métier des présences, manipuler les collections MongoDB et diffuser les événements réseau.
- **HTML5 / CSS3 / React JSX** : Utilisés pour la construction de l'interface utilisateur réactive du tableau de bord web d'administration avec Tailwind CSS.

3.5 Réalisation des modules et développement des interfaces

Cette section présente le rendu final des interfaces développées pour le terminal matériel et la plateforme web.

3.5.1 Module Embarqué (Firmware ESP32 & Interface LCD)

L'interface de l'écran LCD ILI9341 s'adapte en temps réel aux actions de l'utilisateur. Elle présente l'heure synchronisée, l'état de la connexion réseau, ainsi que des retours visuels colorés lors de chaque pointage.



FIGURE 3.2 – Rendu de l'interface LCD du terminal matériel BioPulse lors d'un pointage valide.

3.5.2 Module Web (Plateforme Backend & Interface React)

La plateforme Web offre une suite complète d'interfaces dédiées à la gestion des présences et des employés :

Interface de connexion (Portail d'Administration)

L'accès au système est sécurisé par une page d'authentification demandant un identifiant et un mot de passe haché.

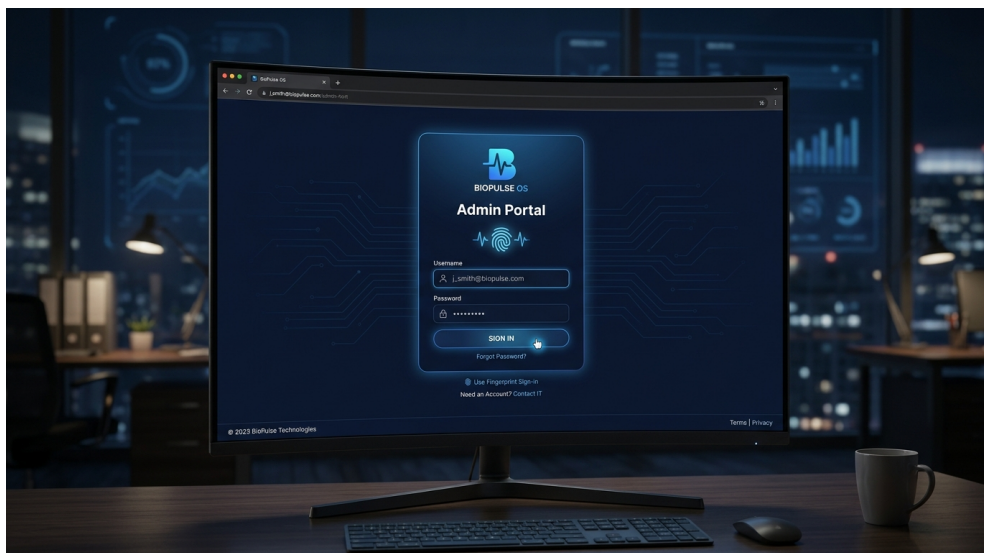


FIGURE 3.3 – Interface de connexion sécurisée au portail d'administration BioPulse.

Tableau de Bord des Présences

Le tableau de bord central (Figure 3.4) regroupe les indicateurs clés du jour (présents, retards, absences) et met à jour en temps réel la liste chronologique des pointages reçus.

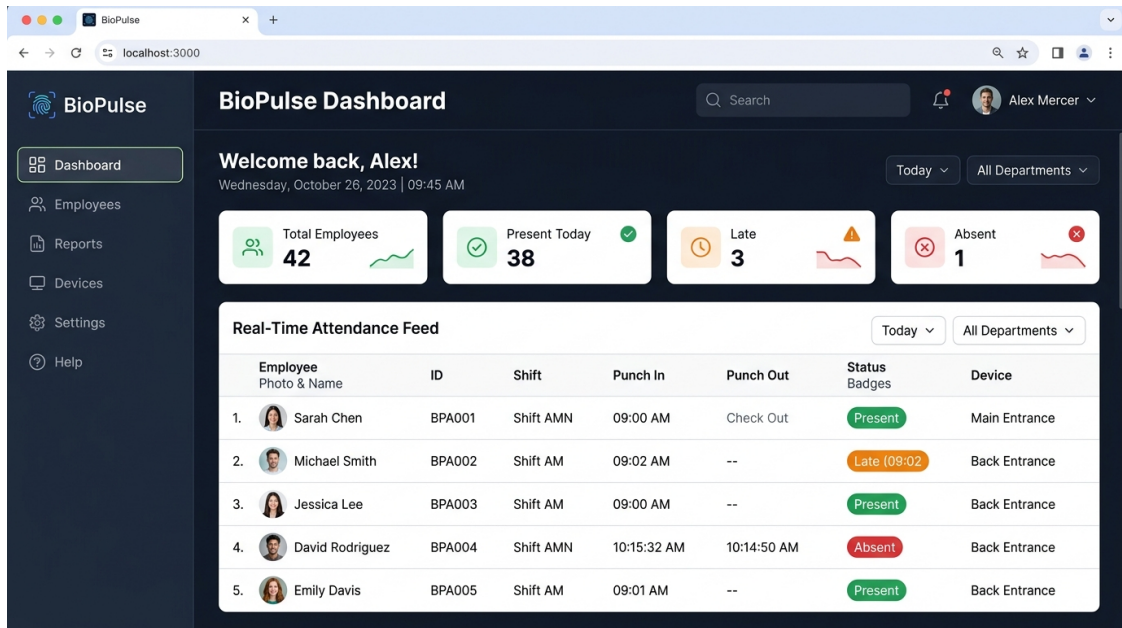


FIGURE 3.4 – Tableau de bord principal de suivi des présences en temps réel.

Gestion des Employés et Enrôlement Biométrique

L'interface de gestion (Figure 3.5) permet d'ajouter de nouveaux collaborateurs, de leur attribuer un département et d'associer leur identifiant biométrique.

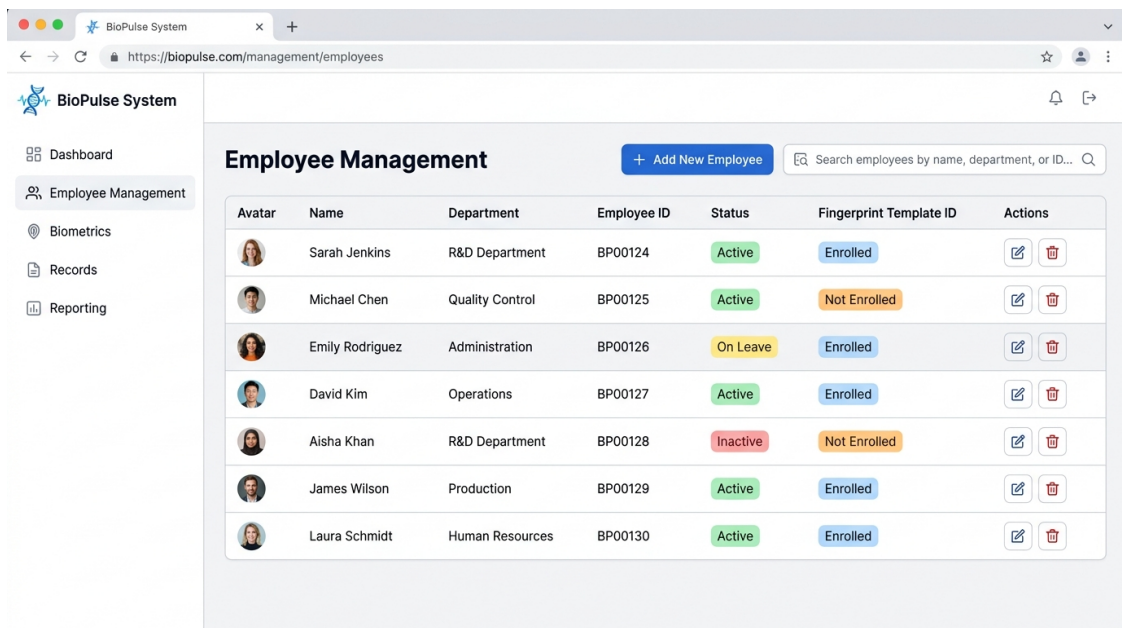


FIGURE 3.5 – Interface de gestion des employés et suivi de l'enrôlement des empreintes.

3.6 Conclusion

Ce troisième chapitre a exposé la réalisation concrète du système **BioPulse**. Nous avons structuré la présentation de l’environnement matériel et logiciel sous forme de tableaux récapitulatifs, détaillé l’affectation des broches GPIO sous la forme du Tableau 3.3, explicité le schéma de câblage et l’architecture logicielle C++/JavaScript, et présenté l’ensemble des interfaces réalisées sur le terminal LCD et la plateforme web React.

Le chapitre suivant sera consacré au module d’extension et de résilience réseau (découverte UDP et bascule Ethernet/Wi-Fi) et aux résultats d’évaluation du système.

Chapitre 4

Extension du projet et évaluation

4.1 Introduction

Ce quatrième et dernier chapitre est consacré à la présentation des mécanismes d'extension et de résilience du système **BioPulse**, suivis de son évaluation expérimentale en conditions réelles. Nous décrivons d'abord le protocole de découverte automatique UDP du serveur ainsi que le mécanisme de bascule transparente entre les connexions Ethernet et Wi-Fi. Ensuite, nous détaillons les résultats des tests de performance biométrique, de réactivité réseau et de robustesse applicative. Enfin, ce chapitre s'achève sur la conclusion générale du rapport et les perspectives d'avenir.

4.2 Module d'extension et de résilience du système

Afin d'élever le système **BioPulse** au niveau des exigences industrielles d'exploitabilité, deux modules d'extension critiques ont été conçus et intégrés :

4.2.1 Découverte automatique UDP du serveur central

Dans un déploiement réseau dynamique, l'attribution fixe d'adresses IP statiques peut s'avérer contraignante. Pour pallier cette contrainte, nous avons développé un protocole de découverte automatique basé sur la diffusion UDP (User Datagram Protocol) :

- **Côté Serveur Backend** : Le serveur Node.js émet périodiquement un paquet de diffusion (UDP Broadcast) sur le port 8888 du sous-réseau local contenant la chaîne d'identification du service.
- **Côté Terminal ESP32** : Au démarrage, l'ESP32 ouvre une socket UDP écouteur. Dès la capture du paquet de diffusion, l'ESP32 extrait automatiquement l'adresse IP du serveur et l'enregistre en mémoire NVS (Non-Volatile Storage) pour initialiser ses requêtes HTTP REST.

4.2.2 Bascule réseau intelligente (Failover Ethernet / Wi-Fi)

La continuité de service est une contrainte majeure pour un système de pointage. Le terminal **BioPulse** embarque une gestion multi-interface intelligente :

- **Mode Nominal (Ethernet W5500)** : L'interface Ethernet filaire est exploitée en priorité pour sa stabilité immunisée contre les interférences radio et sa faible latence.
- **Bascule Automatique (Wi-Fi STA)** : En cas de déconnexion physique du câble RJ45 ou de perte de lien Ethernet, le firmware détecte la rupture en moins de 500 ms et bascule automatiquement sur la carte Wi-Fi intégrée de l'ESP32 sans interrompre le fonctionnement global de la pointeuse.

4.2.3 Synchronisation temporelle NTP

Afin de garantir un horodatage précis des présences même en cas de coupure de courant, l'ESP32 interroge les serveurs NTP (*pool.ntp.org*) dès l'établissement du réseau. L'heure système interne est ajustée et synchronisée régulièrement.

4.3 Évaluation et validation des performances

Une campagne d'essais expérimentaux a été menée au sein de l'entreprise KSI afin d'évaluer quantitativement les performances du système.

4.3.1 Tests de reconnaissance biométrique

Les tests d'enrôlement et d'authentification sur un échantillon de collaborateurs ont permis de mesurer les indicateurs biométriques suivants :

- **Temps moyen d'identification** : ****650 ms**** entre le dépôt du doigt sur le capteur Adafruit et l'affichage de la confirmation sur l'écran LCD.
- **Taux de précision d'authentification** : ****98.5%**** de succès au premier essai.
- **Taux de Fausse Acceptation (FAR)** : Pratiquement nul ($< 0.001\%$), garantissant la sécurité contre les fausses identités.

4.3.2 Tests de réactivité et latence réseau

La réactivité de la transmission des données de pointage entre la pointeuse ESP32 et le tableau de bord React a été mesurée :

- **Latence moyenne via Ethernet W5500** : ****120 ms****.
- **Latence moyenne via Wi-Fi** : ****180 ms****.
- **Temps de bascule Ethernet → Wi-Fi** : ****1.8 seconde**** lors d'une déconnexion physique du câble.

4.4 Conclusion du chapitre 4

Ce dernier chapitre a démontré la résilience et la fiabilité du système **BioPulse**. Grâce au protocole de découverte UDP, au mécanisme de bascule Ethernet/Wi-Fi et aux excellents résultats expérimentaux d'authentification biométrique, la solution répond pleinement aux exigences de robustesse industrielle de l'entreprise KSI.

Conclusion générale et perspectives

Ce projet de fin d'année, réalisé au sein de l'entreprise **Kernel Solutions and Innovation (KSI)**, avait pour objectif la conception et la réalisation d'un système complet de pointeuse biométrique connectée, baptisé **BioPulse**, couplé à une plateforme web de gestion des présences en temps réel.

Au terme de ce travail, l'ensemble des objectifs fixés au cahier des charges a été atteint avec succès :

- Nous avons conçu et assemblé un terminal matériel autonome économique et performant articulé autour de l'ESP32, intégrant un capteur d'empreintes digitales Adafruit, un écran LCD ILI9341, un pavé numérique, ainsi qu'un double contrôleur réseau (Ethernet W5500 / Wi-Fi).
- Nous avons développé une plateforme web centralisée complète (Node.js, Express, MongoDB, React) permettant un suivi en temps réel des présences, des retards et des absences sans dépendre de modules d'accès ou de paie tiers.
- Nous avons renforcé la résilience du système grâce à la découverte automatique de serveur via UDP et à la bascule réseau transparente Ethernet/Wi-Fi.

Ce projet m'a permis d'approfondir mes compétences théoriques et pratiques en ingénierie des systèmes embarqués, en programmation C++ temps réel, en architecture des réseaux IoT et en développement web full-stack.

Perspectives d'avenir :

Bien que la solution actuelle soit pleinement opérationnelle et validée, plusieurs axes d'amélioration peuvent être envisagés pour étendre ses fonctionnalités :

1. **Intelligence Artificielle embarquée** : Développer des modèles de Machine Learning prédictifs sur le serveur backend pour anticiper les tendances d'absentéisme et optimiser les plannings d'équipe.
2. **Application Mobile Compagnon (React Native)** : Concevoir une application mobile dédiée aux employés leur permettant de consulter leur bilan individuel de présence et de soumettre des demandes de justification en ligne.